



Heaps, Homework 7 - Solution

1 Heaps.

1. Is it possible to extract the min/max element of a heap while keeping the heap a heap, in constant time (in the comparison model)? Prove or disprove it.
2. Is the heap sort a stable sort? Prove or give a counter example.
3. Prove that heap construction (Build-Heap) takes $O(n)$ time.

```
input: Array  $H = H_1..H_n$   
1  $i \leftarrow \lfloor \frac{1}{2}n \rfloor$   
2 while  $i \geq 1$  do  
3   | Heapify-down ( $H, i$ )  
4   |  $i \leftarrow i - 1$   
5 end  
6 return  $H_1..H_n$ 
```

Algorithm 1: Build a heap.

4. Given an array where all its elements are different from each other, write an algorithm that returns the k largest elements. Prove its correctness and bound its running time.

Solution.

1. No, assume for contradiction that it is possible. Then one could sort a given array by first heapifying it in linear time and then extracting its elements in order via the 'extract-max' routine. Since extracting costs $O(1)$ time, we have a sorting algorithm in the comparison model that sorts in $\Theta(n)$, which contradicts the fact that any correct sorting algorithm in the model must compare at least $\Omega(n \log n)$ pairs.
2. No, for a counterexample, consider an array consisting of n identical values $A_1 = A_2 = \dots = A_n$. In particular, all the elements satisfy the heap inequality: $A_i \geq A_{2i}, A_{2i+1}$, so the array is already a max-heap.

Therefore, the build subroutine wouldn't change A because the condition at line 10 in the heapify-down subroutine will always be unsatisfied, and no elements will be swapped upon calling it $\Rightarrow$ no swap is going to occur when building the heap.

Yet, in the first iteration of heapsort, the root of the heap, which is also the first element of the array, will be moved to the end of the output array. So there are two equal elements, A_1 and A_2 , such that A_1 is placed before A_2 in the input but placed after it in the output. Namely, heapsort is not a stable sort.

3. A vertex is said to be at height h if the longest path from it to one of the leaves is at length h . Denote by U_h the subset of vertices at height h , using . Also, let $c > 0$ be the constant quantifying the work that is done in each recursive step. Then, we can express the total cost as being bounded from above by:

$$T(n) \leq \sum_{h=1}^{\log n} c \cdot h |U_h|$$

We will need the following claim:

Claim 1. *For any h it holds that:*

$$2^h |U_h| \leq 2n$$

Proof. Recall that the graphical presentation of a heap is an almost complete binary tree. Thus, the complete tree that covers the heap, namely the complete tree that can be obtained by adding leaves to the end of the heap, has less than twice the heap's elements. On the other hand, that tree also covers the union of the h -height subtrees rooted by U_h vertices. The number of vertices in that union sums up to $2^h |U_h|$. Thus, we get the desired inequality $2^h |U_h| \leq 2n$. $\square$

Using the claim we have that:

$$\begin{aligned} T(n) &\leq \sum_{h=1}^{\log n} c \cdot h |U_h| \\ &\leq \sum_{h=1}^{\log n} c \cdot h \frac{2n}{2^h} \\ &\leq 2cn \sum_{h=1}^{\log n} \frac{h}{2^h} \\ &\leq 2cn \sum_{h=1}^{\infty} \frac{h}{2^h} \end{aligned}$$

And that series converges to a constant¹, Hence, $T(n) = \Theta(n)$.

¹By the ratio-test for infinite series, $\lim_{j \rightarrow \infty} \frac{(j+1)2^{-j-1}}{j2^{-j}} = \frac{1}{2}$

4. Consider Algorithm 2. We will prove that it returns the k largest elements in a maximum heap where all its elements are different from each other. Then, we can extract the k largest elements from the array by first heapifying it, and then calling Algorithm 2.

```

1 if  $k = 1$  then
2   | return new-stack( $\{\max H\}$ ), MaxHeap( $\{\emptyset\}$ )
3 end
4 else
5   | Let  $\text{ret}, H_{k-1}^* \leftarrow \text{Call '}(k-1)\text{ largest elements of } H'$ 
6   |  $H_k^* \leftarrow H_{k-1}^*. \text{push}(\{ \text{left}(\text{ret}[0]), \text{right}(\text{ret}[0]) \})$ 
7   |  $\text{ret} \leftarrow \text{ret}. \text{push}(\{ \text{extract-max } H_k^* \})$ 
8   | return  $\text{ret}, H_k^*$ 
9 end

```

Algorithm 2: Find the k largest elements in a heap.

Correctness. For a given heap H , let's denote by V_k the k largest elements in H , by S_k their children, and by x_k the k th element in size. For convenience, let's define V_0, S_0 to be the empty set.

Claim 2. *For a heap where all its elements are different from each other, it holds that $x_k \in S_{k-1}$.*

Proof. First, recall that by definition, x_k is smaller than $k-1$ elements - the elements in $V_{k-1} = x_1, x_2, \dots, x_{k-1}$. Assume for the sake of contradiction that $x_k \notin S_{k-1}$. Thus, x_k is connected to its ancestor in V_{k-1} by at least one element $\notin V_{k-1}$, and by the heap property, it follows that x_k is also smaller than it.

In total, we get that x_k is smaller than k elements, which contradicts the fact that x_k is the k -th element in size. □

Claim 3. *Let H be a heap in which all elements are different from each other, and denote $J = S_{k-1}/V_{k-1}$. Then $x_k \in J$. Furthermore, $x_k = \max J$.*

Proof. From Claim 2, $x_k \in S_{k-1}$. Combining this with the fact that $x_k \notin V_{k-1}$, it follows that $x_k \in S_{k-1}/V_{k-1} = J$.

In addition, any y such that $y > x_k$ is one of the $k-1$ largest items, namely $y \in V_{k-1}$, and therefore $y \notin S_{k-1}/V_{k-1} = J$. Thus, for any $z \in J$, we have that $z \leq x_k$. □

We will prove by induction on k that Algorithm 2 returns a stack of V_k and a heap of S_{k-1}/V_k .

- (a) Base. For $k = 1$, the condition on line (1) is satisfied, and the algorithm returns a stack containing $\max H = x_1$, which is the only element in V_1 . In addition, S_0 is an empty set.
- (b) Assumption. Assume the correctness of the claim for any $k' < k$.

- (c) Step. Consider the running on k . Since $k > 1$, the algorithm skips lines (1)-(3). By the induction assumption, the recursive call to the $k-1$ largest elements returns a stack of V_{k-1} and a heap of S_{k-2}/V_{k-1} , and sets them in ret and H_k^* , respectively.

Thus, after line number (6), which defines the assignment:

$$H_k^* \leftarrow H_{k-1}^*.push(\{\text{left}(ret[0]), \text{right}(ret[0])\})$$

Namely, appending the children of x_{k-1} into the heap so

$$H_k^* = S_{k-2}/V_{k-1} \cup \{\text{children of } x_{k-1}\} = S_{k-1}/V_{k-1}$$

So by Claim 3, $x_k = \max H_k^*$, and therefore after the extraction and assignment in line (7), ret contains $V_{k-1} \cup \{x_k\} = V_k$ and $H_k^* = S_{k-1}/V_{k-1}/x_k = S_{k-1}/V_k$.

Running time. The construction of the heap takes $\Theta(n)$ time. Then, we execute k recursive calls, each performing a constant number of operations and extracting an element from a heap that contains S_{j-1}/V_j , for $j \leq k$, namely from a heap that contains at most $2k$ elements². So overall, the running time is $\Theta(n + k \log k)$.

² $|S_k| \leq 2|V_k| = 2k$